

Projeto 1 — Processamento de Imagens em C com SDL3

Etapa 1 — Análise inicial

Universidade Presbiteriana Mackenzie — Faculdade de Computação e Informática **Curso:** Ciência da Computação · **Disciplina:** Computação Visual **Professor:** André Kishimoto · **Semestre:** 2026.2 **Data:** 08/09/2026

1. Integrantes do grupo

Nome completo	RA
João Guilherme Messias de Oliveira Santos	10426110
Luis Felipe Cunha	10419514

2. Versões das bibliotecas: o que muda em relação ao material da disciplina

O enunciado pede que o projeto use as versões mais recentes e estáveis das bibliotecas SDL, e avisa que o material em vídeo pode estar defasado. Consultamos as páginas de releases dos repositórios oficiais [libsdl-org](https://www.libsdl.org) em 08/09/2026 e chegamos nas versões abaixo.

2.1 Versões que vamos usar

Biblioteca	Versão estável mais recente	Data do release
SDL (SDL3)	release-3.4.16	02/09/2026
SDL_image (SDL3_image)	release-3.4.6	02/09/2026
SDL_ttf (SDL3_ttf)	release-3.2.2	31/03/2025

Duas observações sobre essa tabela:

- O SDL_image mantém duas linhas em paralelo: a [2.8.x](#), que é para SDL2, e a [3.4.x](#), que é a de SDL3. Como o projeto obriga SDL3, a linha certa é a 3.4.x. Vale o cuidado porque a 2.8.12 aparece na listagem de releases junto com as outras e é fácil baixar a errada.
- O SDL_ttf tem o ciclo de releases mais lento das três. A 3.2.2 é a versão estável mais recente da linha SDL3; o que veio depois dela são *prereleases*, e por isso descartamos.

O SDL_ttf não é obrigatório pelo enunciado (aparece como sugestão no item 4), mas pretendemos usar, porque os itens 4, 5 e 6 pedem texto na tela: as informações do histograma e os rótulos dos botões, que mudam conforme o estado.

2.2 O que esperamos ter de mudar no código-base

O código-base foi escrito por outra pessoa e o material da disciplina é anterior a essas versões. Lendo o guia de migração e as páginas de API no site da SDL, identificamos os pontos abaixo como os que provavelmente vão dar trabalho.

a) Retorno das funções. No SDL2 a maior parte das funções retornava `int`, com `0` para sucesso e `-1` para erro. No SDL3 elas retornam `bool`, com `true` para sucesso. Qualquer `if (SDL_Func(...) < 0)` herdado do código-base tem que virar `if (!SDL_Func(...))`. É o tipo de coisa que passa despercebida: o código continua compilando, mas a checagem de erro fica invertida ou nunca dispara.

b) Criação de janela e renderer. `SDL_CreateWindow()` não recebe mais as coordenadas `x` e `y`; a assinatura agora é `(título, largura, altura, flags)`, e a posição é definida depois com `SDL_SetWindowPosition()`. `SDL_CreateRenderer()` também mudou: não recebe mais o índice do driver nem as flags, só `(janela, nome_driver)`. Isso mexe direto no requisito 3 (janela de 1024x768 centralizada) e no requisito 6 (redimensionar e reposicionar a janela principal).

c) Janela filha. A janela secundária tem que ser filha da principal. No SDL3 isso não é uma flag do `SDL_CreateWindow()`: é preciso usar `SDL_CreateWindowWithProperties()` passando a propriedade da janela-pai, ou associar o pai depois. É um dos pontos que achamos menos coberto pelo material introdutório.

d) Desenho de texturas. `SDL_RenderCopy()` virou `SDL_RenderTexture()`, e os retângulos de origem e destino passaram a ser `SDL_FRect` (float) em vez de `SDL_Rect` (int). O mesmo vale para as primitivas que vamos usar nos botões, já que `SDL_RenderFillRect` agora opera sobre `SDL_FRect`.

e) Acesso aos pixels. O campo `format` do `SDL_Surface` deixou de ser um ponteiro para uma struct e virou um valor do enum `SDL_PixelFormat`. Para saber o número de bytes por pixel, as máscaras e os deslocamentos de cada canal — que a gente precisa tanto para a conversão em escala de cinza quanto para o histograma — é necessário chamar `SDL_GetPixelFormatDetails()`. `SDL_MapRGB()` e `SDL_GetRGB()` mudaram de assinatura pelo mesmo motivo. Nossa ideia para simplificar isso é converter a superfície carregada para um formato fixo e conhecido logo depois do carregamento, com `SDL_ConvertSurface()`, e trabalhar sempre em cima dele. Assim não precisamos tratar todos os formatos de entrada possíveis.

f) Inicialização do SDL_image. No SDL2_image era preciso chamar `IMG_Init()` e `IMG_Quit()` com as flags dos formatos. No SDL3_image essas funções foram removidas, basta chamar `IMG_Load()`. Se o código-base ainda chamar `IMG_Init()`, não compila.

g) Assinaturas do SDL_ttf 3. As funções de renderização de texto agora recebem o comprimento da string explicitamente (`size_t length`, com `0` significando string terminada em nulo), e `TTF_OpenFont()` passou a receber o tamanho da fonte como `float`. Qualquer exemplo escrito para SDL2_ttf quebra.

h) Estrutura do `main`. O SDL3 trouxe as *main callbacks* (`SDL_AppInit`, `SDL_AppIterate`, `SDL_AppEvent`, `SDL_AppQuit`) como alternativa ao `main()` com laço de eventos. Optamos por manter o `main()` tradicional, porque o programa precisa receber o caminho da imagem por `argv` e porque controlar o laço na mão facilita separar os eventos das duas janelas. Se o projeto de exemplo que escolhermos já vier com os callbacks, revemos essa decisão.

i) Versão do compilador. O enunciado diz que o projeto será compilado com gcc 15.1.0 no Windows. Nossa máquina hoje tem gcc 14.2.0 (MinGW-W64, UCRT, x86_64-ucrt-posix-seh). Então atualizar o compilador é a primeira tarefa do projeto, antes de escrever qualquer código. Vamos usar a build UCRT do WinLibs, para bater com os pacotes `devel-mingw` oficiais da SDL, que são compilados contra a UCRT.

3. Processo de build

3.1 É o mesmo do material da disciplina?

Não exatamente. O enunciado exige que o código compile com **gcc**, então o processo do grupo é montado em cima do gcc do MinGW-w64 com um `Makefile`, e não em torno de um fluxo de IDE. Onde o material da disciplina usar outro toolchain ou outra forma de trazer a SDL para o projeto, a diferença é essa. O que mantivemos igual foi a ideia central: a SDL entra como dependência externa do projeto e não é misturada ao código-fonte.

3.2 Processo que o grupo adotou

1. **Compilador.** MinGW-w64 na distribuição WinLibs, runtime UCRT, threads POSIX e exceções SEH, com gcc 15.1.0 instalado em `C:\mingw64` e no `PATH`.
 2. **Bibliotecas.** Os pacotes oficiais já compilados para MinGW (`SDL3-devel-<versão>-mingw.zip`, `SDL3_image-devel-<versão>-mingw.zip` e `SDL3_ttf-devel-<versão>-mingw.zip`), extraídos numa pasta `libs/` que fica fora do controle de versão. Preferimos os binários prontos a compilar a SDL do zero via CMake porque isso reduz o número de coisas que podem dar errado no ambiente e faz o build ficar de pé mais rápido. Compilar a SDL a partir do código-fonte continua sendo o plano B.
 3. **Build.** Um `Makefile` na raiz do projeto, chamado com `mingw32-make`, compilando com `gcc -std=c17 -Wall -Wextra` e linkando com `-lSDL3 -lSDL3_image -lSDL3_ttf`. A ideia de concentrar tudo no `Makefile` é que qualquer um dos dois consiga compilar com um comando só, sem depender de configuração do editor.
 4. **Editor.** VS Code nas duas máquinas, com um `tasks.json` que só chama o `Makefile` (sem repetir os comandos de compilação) e um `c_cpp_properties.json` para o IntelliSense achar os headers da SDL.
 5. **Distribuição.** As DLLs (`SDL3.dll`, `SDL3_image.dll`, `SDL3_ttf.dll`) são copiadas para junto do executável como um passo do `Makefile`, porque no Windows elas precisam estar no mesmo diretório do binário.
 6. **Repositório.** Um `.gitignore` tira do versionamento a pasta de build, os `.o`, o executável, as DLLs e a pasta `libs/`. Isso é para não cair na penalidade de -1,0 ponto por enviar arquivos e pastas intermediárias de compilação.
-

4. Sistemas operacionais, compiladores e editores

Os dois integrantes usam a mesma configuração:

Item	Configuração	Versão
Sistema operacional	Windows 11	11 Pro (10.0.26200)
Compilador (instalado hoje)	gcc — MinGW-W64 x86_64-ucrt-posix-seh	14.2.0
Compilador (que vamos usar)	gcc — MinGW-w64 UCRT (WinLibs)	15.1.0
Editor	Visual Studio Code	1.135.0
Controle de versão	Git para Windows	2.53.0.windows.2

Nenhum dos dois tem Linux ou macOS instalado, então o desenvolvimento acontece inteiramente no Windows.

5. Potenciais problemas de configuração

Como os dois integrantes usam a mesma configuração, o enunciado permite desconsiderar este item. Ainda assim vale registrar dois riscos, porque o projeto vai ser compilado numa máquina diferente da nossa na hora da avaliação.

Diferença de versão do gcc. Temos 14.2.0 e o projeto será compilado com 15.1.0. O gcc 15 passou a usar C23 como padrão e transformou em erro algumas coisas que antes eram só aviso, como protótipo implícito e conversão implícita de ponteiro. Ou seja, um código que compila aqui hoje pode não compilar lá. Por isso vamos fixar `-std=c17` no `Makefile` em vez de deixar o padrão do compilador decidir, e compilar com `-Wall -Wextra` desde o começo, resolvendo os avisos em vez de ignorar. Atualizar para o 15.1.0 antes de começar já elimina boa parte desse risco.

Portabilidade para Linux. O enunciado informa que o projeto também será compilado no WSL Ubuntu com gcc 15.2.0. Não temos Linux para testar, então precisamos escrever o código já pensando nisso desde o início. Os dois pontos que mais preocupam:

- *Caminho da fonte.* O requisito 8 pede que a fonte seja carregada corretamente independentemente do sistema operacional. Depender de fonte instalada no sistema não funciona, porque `C:\Windows\Fonts` não existe no Linux e `/usr/share/fonts` não existe no Windows. Vamos incluir o arquivo da fonte no próprio repositório, em `assets/fonts/`, e montar o caminho em tempo de execução a partir do diretório do executável com `SDL_GetBasePath()`.
- *Maiúsculas nos `#include`.* O sistema de arquivos do Windows não diferencia maiúsculas de minúsculas, o do Linux diferencia. Um `#include <SDL3/sdl.h>` compila aqui e falha lá. É só uma questão de conferir a grafia exata dos headers, mas é o tipo de erro que a gente não tem como perceber testando só no Windows.

Também vamos deixar um `.gitattributes` normalizando o fim de linha para LF, para o código não chegar no Linux com CRLF.

6. Itens que conseguimos resolver com o material da disciplina e a documentação da SDL

#	Item do escopo	Nível	Por quê
1	Carregamento de imagem	Confortável	<code>IMG_Load()</code> , checar se o retorno é nulo e usar <code>SDL_GetError()</code> na mensagem. A documentação do <code>SDL_image</code> cobre o caso inteiro, e o tratamento de erro em si é C básico.
2	Deteção de cor e conversão para escala de cinza	Confortável	A fórmula $Y = 0.2125 \cdot R + 0.7154 \cdot G + 0.0721 \cdot B$ está no enunciado. Detectar se a imagem é colorida é percorrer os pixels e verificar se <code>R == G == B</code> em todos. O trabalho está em ler os pixels certo, não no algoritmo. Um de nós já mexeu com transformações ponto a ponto (negativo e alargamento de contraste) nas atividades da disciplina, então a parte conceitual está resolvida.
3	Janela principal 1024x768 centralizada	Confortável	<code>SDL_CreateWindow()</code> mais <code>SDL_SetWindowPosition()</code> com <code>SDL_WINDOWPOS_CENTERED</code> . Está documentado.
3	Janela secundária como filha, em (0,0)	Precisa pesquisar	Ver seção 7.1.
4	Cálculo do histograma	Confortável	Um vetor de 256 posições e uma passada pelos pixels. É contagem de frequência.
4	Desenho do histograma de forma clara e proporcional	Dificuldade parcial	Desenhar as 256 barras com <code>SDL_RenderFillRect</code> é direto. O problema é a normalização: um pico muito alto (um fundo uniforme, por exemplo) achata visualmente todo o resto do gráfico. Temos que decidir entre normalizar pelo maior valor, cortar percentis ou usar escala logarítmica. É decisão de visualização, não de API.
4	Média, desvio padrão e classificação	Confortável	As duas contas saem direto do histograma. O que exige critério é escolher onde cortar as faixas de "clara / média / escura" e de contraste "alto / médio / baixo". Vamos definir esses limiares e justificar no relatório final.
5	Botões com primitivas e com estados visuais	Confortável	Um retângulo, um teste de ponto dentro do retângulo com a posição do mouse e três cores conforme o estado. Não precisa de nada além de <code>SDL_RenderFillRect</code> e dos eventos de mouse.
5	Alternar entre equalizada e original sem recarregar	Confortável	Guardar as duas versões em memória (a original em cinza e a equalizada) e trocar qual está sendo exibida. É gerenciamento de memória, que é conteúdo de C.
5	Algoritmo de equalização	Precisa pesquisar	Ver seção 7.2.

#	Item do escopo	Nível	Por quê
6	Alternar resolução e redimensionar a janela	Dificuldade parcial	<code>SDL_SetWindowSize()</code> e <code>SDL_RenderTexture()</code> com destino escalado resolvem a maior parte. A regra condicional de posicionamento é que complica. Ver seção 7.4.
7	Salvar imagem com a tecla <code>S</code>	Confortável	<code>IMG_SavePNG()</code> e um evento de teclado. Sobrescrever já é o comportamento padrão; para escolher entre a mensagem de "criado" e a de "sobrescrito", basta verificar se o arquivo existe antes de salvar.
8	Carregar a fonte independentemente do SO	Dificuldade parcial	A API do <code>SDL_ttf</code> é simples. O problema é decidir qual fonte usar e como distribuí-la, não como chamar a função. Ver seções 5 e 7.5.

7. Itens que vamos ter que pesquisar mais a fundo ou pedir ajuda de IA generativa

7.1 Janela secundária como janela filha (requisito 3)

É o item em que temos menos base. A documentação da SDL descreve a propriedade de janela-pai no `SDL_CreateWindowWithProperties()`, mas não deixa claro o que acontece na prática: se a janela filha fica sempre por cima da principal, se aparece na barra de tarefas, se fecha junto quando a principal fecha. Além disso, trabalhar com duas janelas significa distinguir de qual delas veio cada evento pelo `windowID`, e isso não aparece nos exemplos de janela única.

7.2 Equalização de histograma (requisito 5)

É o item que mais vale ponto na avaliação do software (2,0 pontos, sendo 1,5 só para a equalização estar certa) e é o único requisito que é de fato um algoritmo de processamento de imagens, não uso de API. Sabemos a ideia geral, que é redistribuir as intensidades usando a função de distribuição acumulada (CDF) do histograma para espalhar os tons pela faixa toda, mas nenhum de nós implementou ainda. O que precisamos estudar:

- a fórmula exata do mapeamento, incluindo o tratamento do menor valor não nulo da CDF (`cdf_min`), que é o que evita a imagem sair deslocada;
- a aritmética inteira: arredondamento e risco de estouro ao multiplicar a CDF acumulada por 255 em imagens grandes;
- a diferença prática entre equalização, alargamento de contraste e correção gama. As três atacam o mesmo problema, mas só a equalização usa a distribuição real da imagem. Entender isso é o que permite justificar o resultado, em vez de só mostrar que a imagem mudou.

7.3 Manipulação de pixels em formatos diferentes

Isso não é um item do escopo, mas é a base dos requisitos 2, 4, 5 e 7. Se a leitura dos pixels estiver errada, o tom de cinza, o histograma, a equalização e a imagem salva ficam todos errados juntos. As dúvidas

concretas são: calcular o endereço de um pixel usando o `pitch`, que não é simplesmente largura vezes bytes por pixel por causa do alinhamento de linha; quando é necessário chamar `SDL_LockSurface()`; e a ordem dos canais em cada formato. A ideia de converter tudo para um formato fixo logo no carregamento (seção 2.2, item "e") foi pensada justamente para diminuir esse risco, mas ainda precisa ser testada.

7.4 Regra de posicionamento condicional da janela (requisito 6)

Exige consultar a resolução do monitor principal em tempo de execução e decidir entre centralizar a janela ou fixá-la em (0,0). É um item pequeno, mas depende de APIs de display que não costumam aparecer em material introdutório, e que se comportam de forma diferente em monitores com escala de DPI alta. Isso pode fazer a janela parecer descentralizada mesmo com a conta certa.

7.5 Escolha e empacotamento da fonte (requisito 8)

Aqui a dificuldade não é técnica. É escolher uma fonte cuja licença permita redistribuir o arquivo junto do código num repositório público. Pretendemos usar uma fonte de licença aberta, da família DejaVu ou Liberation, colocar em `assets/fonts/` e carregar por caminho relativo ao executável. Conferir a licença exige olhar fora da documentação da SDL.

8. Resumo

Situação	Itens
Confortável	1 (carregamento), 2 (escala de cinza), 3 (janela principal), 4 (cálculo do histograma e estatísticas), 5 (botões e alternância sem recarregar), 7 (salvar imagem)
Dificuldade parcial	4 (desenho proporcional do histograma), 6 (redimensionamento e posicionamento), 8 (fonte independente de SO)
Precisa pesquisar a fundo	3 (janela filha), 5 (algoritmo de equalização), manipulação de pixels em formatos diferentes

9. Próximos passos

1. Atualizar o compilador para o gcc 15.1.0 (WinLibs UCRT) nas duas máquinas e conferir com `gcc --version`.
 2. Baixar os pacotes `devel-mingw` da SDL3 3.4.16, SDL3_image 3.4.6 e SDL3_ttf 3.2.2.
 3. Escolher qual projeto de exemplo do repositório da disciplina vai servir de base.
 4. Escrever o `Makefile` e validar com um programa mínimo que só abre uma janela.
 5. Implementar os requisitos na ordem 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8, dividindo as tarefas entre os dois e fazendo commits pequenos e frequentes, para atender ao critério de atualizações constantes no repositório.
-

10. Referências

- Enunciado do Projeto 1 (Proj1) — Computação Visual, Prof. André Kishimoto, 2026.2.
- Repositório de exemplos da disciplina (Moodle).
- Documentação oficial da SDL — <https://wiki.libsdl.org/SDL3/>
- Guia de migração SDL2 → SDL3 — <https://wiki.libsdl.org/SDL3/README-migration>
- Páginas de releases oficiais de [libsdl-org/SDL](#), [libsdl-org/SDL_image](#) e [libsdl-org/SDL_ttf](#), consultadas em 08/09/2026.